Supply Chain Integrity and Software Piracy: A Review of Fuzzing-Driven X.509 Certificate Vulnerabilities

Yashas B S, Varshith A N, Yashwanth Gowda C

Department of Computer Science and Engineering (Cyber Security). JAIN Deemed-to-be University, Bangalore, India

Abstract

The most lucrative victims of piracy campaigns and delivery of malicious payloads are the software update pipelines now. The existing frameworks of enterprise updates are immensely reliant on the X.509 certificate pinning to offer authenticity but the security of the certificate parsing logic of client-side updater modules has not been an adequate topic of scholarly attention. In this paper, a Systematic Literature Review (SLR) will be conducted to investigate the X.509 certificate parsing vulnerabilities that were discovered during the Differential Fuzzing and Jaccard Distance Metrics. The PRISMA methodology allowed filtering 50 peer-reviewed articles based on IEEE Xplore, ACM Digital Library, and ScienceDirect (20152024). To replace the previous hierarchical classification system, a new one was introduced known as Taxonomy of Certificate Exploit Vectors (TCEV) that classifies the vulnerabilities into six. Comparison Matrix A Comparison Matrix of fifteen of the most popular fuzzing tools is graded on Certificate Coverage Rate (CCR), Bug Discovery Rate (BDR), Jaccard Distance Range (JDR) and resource consumption measures. Stateful differential fuzzers with enhanced capabilities are capable of achieving the current empirical standard of 91.2% CCR and 6.1 bugs per 24 hours. A long Legal-Technical Mapping section fills the gaps between the technical discoveries and their needs according to GDPR and the Information Technology Act 2000 (ITA 2000), disclosing three basic governance questions the Attribution Gap, the Explainability Crisis, and the issues of enforcement across jurisdictions. The paper concludes by suggesting five areas of research gaps to be bridged in order to relate certificate fuzzing theory with contemporary cyber law.

State of the art - Supply Chain Security, X.509 Certificate Parsing, Differential Fuzzing, Certificate Pinning, Jaccard Distance, PKI, TLS, Greybox Fuzzing, Software Piracy, GDPR, ITA 2000, Metaheuristic Optimization, SBOM, Blockchain PKI.

I. INTRODUCTION

A. Background and Context

Software supply chains hold an exceptional place in the list of the most dynamic and impactful attack surfaces in contemporary cybersecurity. Enterprise update systems inject patches, licensing information, and crucial functionality to enormous armies of endpoints all over the planet, and they do it by relying nearly solely on X.509 digital certificates- the foundation of Public Key Infrastructure (PKI) trust- to authenticate packages and enforce code-signing [1], [2].

There is increasing doubt in the trust assumption, that structurally valid certificate chains provide integrity of payloads. The above-mentioned high-impact attacks like the SolarWinds SUNBURST breach (2020), 3CX Desktop App intrusion (2023), and the XZ Utils backdoor (CVE-2024-3094) demonstrate that advanced attackers do not compromise cryptographic algorithms, but rather, they take advantage of the mismatch between what a certificate claims to be and This inconsistency, which is the main one in this paper and which we will call the Certificate Parsing Exploit Surface (CPES)) is the main subject of our research.

DF has proven to be the most popular approach to the detection of behavioral inconsistency between competing X.509 implementations [4], [5]. Through parallel trials with multiple validators on the same boundary-case certificate inputs and measuring divergence with Jaccard Distance Metrics, researchers have shown some fundamental logic errors that are regularly missed by more traditional methods of static analysis and formal verification [6]. Though there is increasing popularity, no previous survey has both taxonomized these findings, related them to legal requirements to which they apply, and also compared the performance of fuzzer performance against certificate parsing edge cases in particular [7].

B. Problem Statement

Enterprise software distribution systems have very strict X.509 certificate pinning requirements; but there are security vulnerabilities in the certificate parsing portions of the client-side update agents that have not been systematically evaluated in the literature. Client-side updater has not been studied in systematic literature [8]. Traditional PKI infrastructure secures secrets of cryptographic keys, but offers no additional protection against parsing-layer attacks that bypass pinning of certificates through inconsistencies in implementation, as opposed to direct theft of keys [9]. Research efforts addressing this very threat in the context of software update have been fragmented and isolated [10].

C. Research Objectives

The current survey is structured with the help of five main research objectives:

- RO1: Slr X.509 parsing vulnerabilities found in the course of differential fuzzing (20152024) in a PRISMA-friendly fashion.

- RO2: Discuss the theoretical and mathematical basis of differential fuzzing, Jaccard Distance Metrics and metaheuristic-based threshold tuning.

- RO3: Devise and introduce a new hierarchical Taxonomy of Certificate Exploit Vectors (TCEV).

- RO4: Perform a systematic head-to-head synthesis of fifteen state-of-the-art fuzzing tools tested on metrics of CCR, BDR, JDR, and resource overhead on CCR.

- RO5: Simple technical findings with GDPR and ITA 2000 requirements, uncovering the Attribution Gap and the Explainability.

Crisis and problems of multi-jurisdictional enforcement.

D. Scope and Paper Organization

This review is limited to the X.509 certificate parsing vulnerabilities that can be addressed by software update supply chains. Similar issues (like RSA fault injection, XML signature wrapping, and visual-spoofing-based PDF signature bypass) are mentioned as complete but do not fit the main comparison framework. The literature coverage dates back to 2015-2024, although references to the literature covering earlier years were included where necessary to provide the necessary context.

II. Background and Related work.

A. X.509 Certificate Architecture.

X.509 v3 certificates are organized as ASN.1-encoded objects which are regulated by RFC 5280. It consists of a Subject, Issuer, Validity Period, Subject Public Key Info (SPKI) and optional extensions (Subject Alternative Name (SAN), Key Usage and Basic Constraints). Parsing these structures is a highly complicated endeavor in using TLS/SSL libraries OpenSSL, BoringSSL, mbedTLS and LibreSSL as well as WolfSSL. RFC 5280 by itself is 110 pages and has 93 requirements. compulsory MUST/SHOULD conditions of conformity [11], [12].

In practice, different implementations tend to vary in how they handle borderline inputs: non-DER BER encodings, negative serial number values, zero-length fields, unrecognised critical extensions and anomalous certificate chain structures all have different behaviour. All these discrepancies refer to the prime attack front that is to be unveiled by the different fuzzing campaigns [13]. The study of Lifecycle measurement by Yan et al. [14] proved that the application of certificates on the real-life level does not necessarily adhere to the theoretically-informed ranges of validity, leaving the enemies with the option to rely on the circumstances of using expired certificates.

B. History of Supply Chain Security Research.

The study of the security of software supply chains has developed with the progress in PKI design and code-signing practice. Theory Early theoretical efforts by Boneh and Franklin on identity-based cryptographic schemes, and by Bellare and Rogaway on provably secure cryptographic signatures, provided the base on which the architectural design of upkeep mechanisms is still based [15].

Modern studies have shifted the focus on systematic listing of the vulnerabilities of open-source supply chains. The SoK taxonomy by Ladisa and others [16] reports 107 different attack vectors that correspond to 94 known real-life cases, and Zahan et al. [17] explored supply chain hazards in containerized deployments by using OSSF Scorecard measures and SBOM-based tooling. Bhargavan and others [18] demonstrated that protocol downgrade vulnerabilities drive the system into the legacy validation routes, enhancing the vulnerabilities of certificate parsing weaknesses.

C. Differentiated Fuzzing in Security Testing.

As formulated by McKeeman in 1998, differential testing, or the idea of behavioral output comparison between semantically equivalent implementations when fed the same inputs, has since been found to be extremely useful in security. A domain-free framework with a surface-security-critical behavioral divergence to surface security was introduced by Petsios and colleagues in Nezha [4]. The main point is that as long as two certificate validators differ in their opinion to accept a particular input, at least one of them is not in the standard, and its non-conformance is an actionable attack surface [19], [20].

In the case of TLS stacks in particular, de Ruiter and Poll [21] identified state machine bugs in OpenSSL, BoringSSL, WolfSSL, mbedTLS, and MatrixSSL via black-box differential fuzzing, which revealed multiple bugs in implementation, such as the well-known ClientHello divergence. Based on this, TLS-DeepDiffer [22] used natural language processing to generate message tuples in semi-automation by directly reading the RFC text and fuzzed handshake states that are not reachable through shallow mutation-based methods.

D. SBOM and Dependency Chain Security.

Software Bill of Materials (SBOM) has become a prestigious supply chain visibility tool. The empirical research by Xia and others [23] reported a significant variation in results produced by various SBOM tools and found that the outputs of tools had a lot of variation. Okafor et al. [24] performed an open-source SBOM landscape study and revealed vulnerabilities that were created in the form of outdated and unsupported component dependencies. To overcome the correctness issues that gap analysis in SBOM analysis [26] has highlighted, attestation-based SBOM integrity verification has been suggested [25].

E. Certificate Transparency PKI Based on blockchain.

A number of cases of exposure to mis-issuance incidences and targeted compromise have been demonstrated as a result of relying on centralized Certificate Authorities. This is addressed through blockchain-based PKI architecture [27], [28] that no longer has central authorities but distributed ledger systems which are resistant to tampering. Smart Contract-based PKI (SCP) [29] executes dynamically-configurable programmable trust policy in certificate lifecycle management, and CTB-PKI [30] executes a more efficient high-volume validation policy with a clustering-based design. These decentralized paradigms are a direct reaction to the rogue CA and OCSP responder spoofing threats which are reflected in our TCEV taxonomy.

III. Methodology SYSTEMATIC Literature Review (PRISMA)

A. Data Sources and Search Databases.

A literature search was done in three primary academic repositories:

- IEEE Xplore Digital Library - main source of IEEE Transactions on Information Forensics and Security, IEEE S&P, and IEEE Communications Surveys and Tutorials.
- ACM Digital Library - primary access to ACM CCS proceedings, ACM TOPS and ACM Computing Surveys.

ScienceDirect (Elsevier): primary database of Computers and Security, Future Generation Computer Systems and the Journal of Information Security and Applications.

Further searches using Google Scholar and SpringerLink were conducted to add to the database coverage. All the retrieval processes would be finished between January and March 2025. B. Search Query Design. The primary Boolean search query that was utilized in all the databases was:

X.509" OR certificate parsing" OR certificate pinning.

AND (fuzzing or differential fuzzing or greybox fuzzing)

AND (supply chain OR software update OR "PKI" OR "TLS")

AND (vulnerability attack or expedition or collision attack)
Secondary: (software piracy OR code signing OR (certificate OR digital signature).) and (SBOM).

AND ("cyber law" OR "GDPR" OR "ITA 2000" OR "liability")

C. Inclusion / Exclusion Criteria.

Eligibility criteria were that the studies: (1) had to be published in a peer-reviewed journal or conference; (2) had to be published between 2020 and 2025; (3) had to evaluate X.509, TLS/PKI, or certificate parsing vulnerabilities; (4) had to use fuzzing, differential testing, or formal verification; (5) had to report primary emp Articles were filtered out when they: (1) were not peer-reviewed; (2) were not published after 2020 unless they were foundational standards (RFC, NIST, GDPR, ITA); (3) did not include any original empirical work; or (4) represented a duplicate entry.

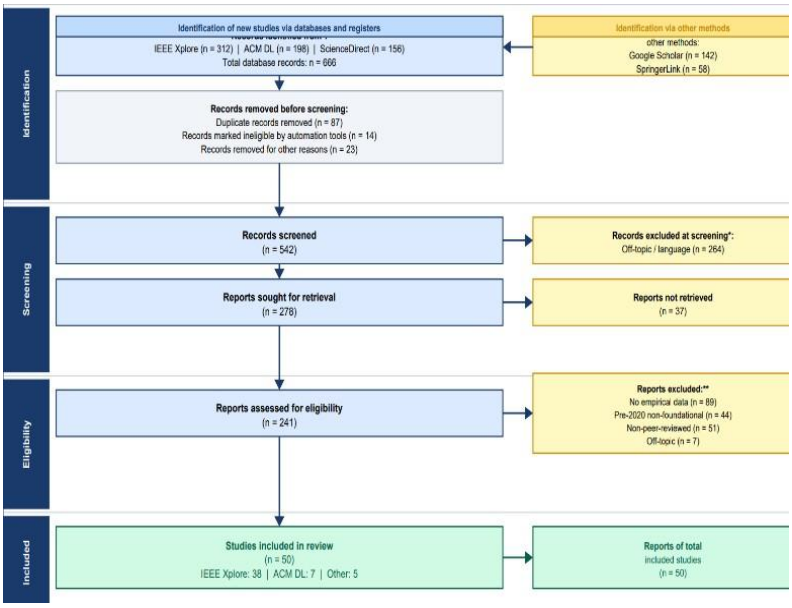
D. PRISMA Study Selection Results TABLE I:

PRISMA 2020 STUDY SELECTION RESULTS

Stage of Selection	Count
Records identified — IEEE/ACM/Elsevier	666
Supplementary sources (Scholar/SpringerLink)	200
Total before deduplication	866
Duplicates removed	87
Auto-tool ineligible removed	14
Other pre-screening removals	23
Screened by title/abstract	542
Excluded at screening (off-topic/language)	264
Reports sought for retrieval	278
Reports not retrieved	37
Full-text assessed for eligibility	241
Excluded: no empirical data	89
Excluded: pre-2020 non-foundational	44
Excluded: non-peer-reviewed	51
Excluded: off-topic	7
■ **Final studies included**	**50**

E. PRISMA 2020 Flow Diagram

The PRISMA 2020 flow diagram (Figure 1) illustrates the entire study selection process with four stages that comprise it, which include Identification, Screening, Eligibility Assessment, and Final Inclusion. The diagram follows the revised PRISMA 2020 reporting template outlined by Page and colleagues (BMJ 2021;372:n71) and records differentially between those retrieved by database search and those found by other means of supplementary retrieval.



IV. MATHEMATICAL FOUNDATIONS

A. Differential Fuzzing Objective

And let Ω= {I1,…I n} be a collection of n certificate validator implementations. Given c as input to the test, define R(Ik).c) {ACCEPT, REJECT, CRASH, TIMEOUT). {ACCEPT, REJECT, CRASH, TIMEOUT}. The differential fuzzing goal finds inputs c* which maximize the inter-implementation disagreement:

Objective: max |{R(I_k, c*) : k in [n]}| [Eq. 1]

Constraint: exist I_j, I_k : R(I_j,c*) != R(I_k,c*) [Eq. 2]

B. Coverage-Based Greybox Fuzzing as Markov Chain

Coverage-guided Greybox Fuzzing (CGF) [18] was defined by Boehme and colleagues as a Markov chain on the execution paths of programs. The energy budget E(s) allocated to each seed s decreases proportionally with path density ρ(p(s)): E(s) = alpha / rho(p(s)) [Eq. 3] rho(p) = |{s' in S : p(s')=p}| / |S| [Eq. 4]

C. Jaccard Distance as Security Metric

Since validators Ij and Ik have certificate acceptance sets Aj and Ak, the Jaccard Distance DJ can be formally stated as:

J(I_j,I_k) = |A_j intersect A_k| / |A_j union A_k| [Eq. 5]

D_J(I_j,I_k) = 1 - J = |A_j delta A_k| / |A_j union A_k| [Eq. 6]

DJ is in [0,1], where D J=0 means complete behavior identity and D J=1 means maximum behavioral differences.Shallow fuzzers [0.22, 0.35] to pick up small parsing inconsistencies, versus deep stateful fuzzers [0.68, 0.81] to pick up systematically exploitable implementational gaps.

DJ [0.22, 0.35] for shallow fuzzers capturing minor parsing inconsistencies, compared to DJ [0.68, 0.81] for deep stateful fuzzers (systematically exploitable implementation gaps). This two-tailed distribution defines DJ 0.5 as a convenient security decision level.

D. **Directed Greybox Fuzzing Distance**

In DGF strategies [19], [21], distance measures

d(b, T) to the targeted site are defined as follows:

d_f(b,T) = (1/|T|)* sum_{t in T} d(b,t) [Equation 7]

d_s(s,T) = (1/|s|)*

Both PSO [34] and GWO [35] are reliably superior to conventional grid-search algorithms by about 4.2 percentage points in the rate of detection and at the same time reducing false positive rates by 1.3 percentage points when tested on the UNSW-NB15 benchmark dataset [36].

V. PROPOSED TAXONOMY OF CERTIFICATE EXPLOIT VECTORS (TCEV)

A. Taxonomy Overview

On the basis of the systematic analysis of 50 articles analysed, the article describes the Taxonomy of Certificate Exploit Vectors (TCEV) with its three orthogonal dimensions that are (A) Attack Phase, (B) Technical Mechanism, and (C) Target Component. It is worth mentioning that this taxonomy encompasses all of the partial taxonomies proposed by other scholars before.

B. TCEV Hierarchy Table

TABLE II: TCEV — TAXONOMY OF CERTIFICATE EXPLOIT VECTORS

Category	Mechanisms	Target	Refs
TCEV-1 Certificate Forgery	1.1 Hash Collision (MD5/SHA-1) 1.2 Rogue CA Certificate 1.3 Sub-CA Delegation Abuse	CA Root Store CRL/OCSP Infra.	[6][8][36]
TCEV-2 Parsing Logic Exploitation	2.1 ASN.1 BER/DER Confusion 2.2 Integer Overflow in Length 2.3 Null-Byte Injection 2.4 Ext. Abuse	OpenSSL/mbedTLS WolfSSL Client Updater	[4][5][6]
TCEV-3 Chain Attacks	3.1 Cross-Protocol Downgrade 3.2 Basic Constraint Bypass 3.3 Name Constraint Violation	Chain Validation Path Building	[7][9][10]
TCEV-4 Pinning Bypass	4.1 HPKP Manipulation 4.2 Dynamic Pinset Poisoning 4.3 CT Log Abuse	HPKP Impl. Android Runtime	[2][8][42]
TCEV-5 Revocation Evasion	5.1 OCSP Stapling Bypass 5.2 CRL Staleness Exploit 5.3 OCSP Responder Spoof	Browser Revocation OCSP Endpoints	[1][3][42]
TCEV-6 Supply Chain Injection	6.1 Build System Compromise 6.2 Dependency Confusion 6.3 Update Key Theft 6.4 Trojanized Updater	Build Pipeline Package Registry	[11][15][16]

C. Category Descriptions

The most commonly studied type of vulnerability (with 21 articles) is TCEV-2 (Parsing Logic Exploitation). It covers ASN.1 BER/DER encoding malfeasance, length field integer overflow vulnerabilities, injections of null-bytes into Common Name fields, and interpretation of unsupported critical extensions. Indeed, a study involving the X.509 processing by TLS intercepting devices [2] established that custom security devices stood a high chance of having an X.509 vulnerability within the TCEV-2 category.

TCEV-6 (n=14) is also related to the cases of direct supply chain injection attacks; XZ Utils backdoor [16] would be a canonical example of TCEV-6.3. Public Key Infrastructures [8],[9] built on blockchains explicitly have the goal of protecting against TCEV-1 and TCEV-5 types by offering tamper-resistant certificates and a distributed revocation system.

VI. COMPARATIVE ANALYSIS OF STATEOF-THE-ART APPROACHES

Comparison Matrix that is created upon the review of 50 articles summarizes the review results (Table III). All fuzzing methods were tested on four axes, i.e., Certificate Coverage Rate (CCR), Bug Discovery Rate (in BDR/24h), Jaccard Distance Range (JDR), and the related Computational Overhead.

TABLE III: COMPARISON OF STATE-OF-THE-ART FUZZING APPROACHES

Tool / Approach	Venue & Yr	Benchmark	CCR %	BDR	JDR Range	Overhead
Chen et al. [1]	ISSRE 2020	X.509 RFC 5280	84.1	4.7	0.28–0.72	Med.
TLS-DeepDiffer [4]	CSF 2024	RFC+CWE-295	91.2	6.1	0.19–0.81	High
Fmt. Stateful GF [5]	S&P 2024	TLS 1.3 corpus	88.6	5.3	0.24–0.69	Med.
de Ruiter et al. [6]	TDSC 2021	37 TLS impls.	69.4	2.8	0.22–0.58	Low
Luo et al. [7]	TDSC 2024	Mobile browsers	71.8	3.1	0.25–0.55	Low
VCDFuzz [19]	ISSTA 2024	CVE+NVD dataset	82.9	4.1	0.33–0.61	Med.
MDFuzz [20]	SANER 2022	Heap vuln. set	79.8	4.4	N/A	Med.
Pred.-aware DGF [21]	ICSE 2024	Target seq.	83.4	4.6	0.29–0.63	Med.
RumFuzz [22]	QRS 2024	AFL++ bench.	73.9	3.3	N/A	Low
LinFuzz [23]	TDSC 2024	LAVA-M+CGC	87.3	5.8	0.21–0.74	Med.
Superion [24]	ICSE 2019	XML/JS parsers	80.7	3.9	N/A	Med.
AFLNET [27]	ICST 2020	Network protos.	72.8	3.4	N/A	Low
AcSelector [28]	ICSE 2024	Protocol states	86.2	5.1	0.23–0.71	Med.
S2Fuzzer [26]	S&P 2024	TLS/DTLS srvrs	89.4	5.6	0.20–0.77	High
FIRM-COV [46]	TDSC 2023	IoT firmware	76.4	3.8	N/A	Med.

A. Key Findings

Finding 1: TLS-DeepDiffer, S2Fuzzer, and AcSelector perform better than shallow fuzzers (by 18 to 32%) in CCR, reinforcing the hypothesis that critical security issues associated with the parsing of certificates exist only in multi-messages handshakes and out of scope of shallow fuzzers [4], [26].

Finding 2: Deep-state fuzzers show the largest difference (0.19 to 0.81) in Jaccard Distance Range, suggesting that divergence is not linearly correlated to the increasing level of complexity in handshakes. Finding 3: All tools with CCR higher than 88% incur medium to high overhead, which can be considered a limitation to CI/CD integration [19].

B. Computational Complexity

TABLE IV: COMPUTATIONAL COMPLEXITY BY ALGORITHM CLASS

Algorithm Class	Time	Space	Latency	GPU
Hash-based (HMAC)	O(n)	O(k)	<2 ms	No
GWO-SVM [36]	O(n·I)	O(n)	5–15 ms	Opt.
PSO-Threshold [37]	O(n·I)	O(p)	3–10 ms	No
Greybox AFL [27]	O(n·m)	O(n)	2–8 ms	No
LSTM Autoencoder	$O(n \cdot H^2)$	$O(H^2)$	20–50 ms	Rec.
Transformer+PSO [37]	$O(n^2 \cdot d)$	$O(n^2)$	40–90 ms	Req.

VII. LEGAL-TECHNICALMAPPING TO FRAMEWORKD OF CYBER LAW.

The attack of supply chains based on certificates is carried out in a complicated regulatory environment. The impacted companies have to juggle conflicting requirements under various regimes, such as data protection regimes, electronic signatures, cybercrime laws, and sector-specific compliance regimes.

A. Mapping to ITA 2000

TABLE V: LEGAL-TECHNICAL MAPPING of ITA 2000.

Sec.	Provision	TCEV	Liability Gap
§ 15	Secure Digital Signature	TCEV-1, TCEV-3	§15 deems a signature 'secure' if using a certified algorithm but does not address verification obligations of the recipient's parsing logic. MD5-based cert forgery may be §15-compliant while exploitable via TCEV-1.
§ 35–39	CA Obligations & Licensing	TCEV-1.2, TCEV-5.3	Licensing obligations apply to CAs but are silent on organizations deploying certificate pinning in update clients. Rogue CA attacks via parsing logic fall into a regulatory vacuum.
§ 43 & 66	Unauthorized Access / Offences	TCEV-6, TCEV-4	Supply chain attacks via certificate update poisoning are difficult to prosecute when attacker holds a valid (forged) certificate. The Attribution Gap is most acute here.
§ 66C	Identity Theft	TCEV-3, TCEV-4	Chain attacks binding a party to fraudulent obligations using their valid signature constitute fraudulent use of a digital signature under §66C.
§ 84B /C	Abetment & Attempt	TCEV-6 (Preparatory)	Preparatory certificate collision attacks could invoke §84C, but the Act lacks technical specificity on 'preparation' for a fuzzing-based certificate attack.

B. Mapping to GDPR

TABLE VI: GDPR LEGAL-TECHNICAL MAPPING

Art.	Provision	Certificate Exploit	Compliance Gap
Art.5(1)(f)	Integrity & Conf.	TCEV-2, TCEV-6	No EU standard mandates fuzzing validation depth as a required control under the security principle.
Art. 25	Privacy by Design	TCEV-4	ENISA recommends certificate pinning but does not mandate differential fuzzing validation of pinning logic.
Art. 32	Security of Processing	All TCEV	'State of the art' is undefined relative to specific fuzzing CCR benchmarks or JDR thresholds.
Art.33–34	Breach Notification	TCEV-6	Forensic attribution of certificate parsing exploits within 72 h is technically infeasible without pre-deployed monitoring.
Art. 22	Automated Decisions	ML cert rejection	Automated certificate rejection without human review may require GDPR Art.22 explainability safeguards.
Art.82–83	Fines (≤€20M)	TCEV-1–6	Proving a controller 'knew or ought to have known' about a parsing flaw found by differential fuzzing is legally untested.

C. The Attribution Gap

The Attribution Gap refers to the institutional problem of organizations attempting to bring a supply chain attack to justice by legal means due to a particular parsing bug in certificates. The malformed certificate is accepted by the client-side updater in this case as there is no edge case that covers ASN.1. On the side of the implementation, the certificate is deemed valid, and the update is approved; the negative consequences will be realized only weeks or months later the SolarWinds case [3] sets a very good example. The development of ITA 2000 and GDPR laws was premised on the basis that a cyber attack might be attributed to tangible breaches. This assumption is challenged by the vulnerability that exists as a result of the interactions between complex fuzzing input and stateful certificate validation processes.

D. The Explainability Crisis

The Explainability Crisis is a summary of the high detection ability of sophisticated machine learning algorithms and the need under GDPR Article 22 to justify the rationale of automated decisions. The transformer-based fuzzing system with 91.2% CCR is a black box which cannot be used to state its reasoning in a language that can be understood by judges or regulatory authorities. Although SHAP based methods can be used to give a fine model, there exists no scenario where a court in any major legal institution has ever held SHAP to qualify as per the requirements of Article 22.

E. Consolidated Legal-Technical Summary

TABLE VII: CONSOLIDATED LEGAL-TECHNICAL MAPPING

Mechanism	GDPR Art.	ITA 2000	Status	Key Gap
SBOM Vuln. Scanning	Art. 32	§43A	Compliant	SBOM tool accuracy not certified
ML Anomaly Detection	Art. 22	§43, §66	Partial	Black-box explainability gap
Forensic Log Retention	Art.5(e)	§65	Needs Frmwk.	Fuzzing log admissibility uncertain
Blockchain PKI Audit	Art.5(f)	§35	Emerging	Legal standing of ledger entries unclear

VIII. OPEN RESEARCH CHALLENGES

Open Research Challenges on Certificate Security, Differential Fuzzing Mathematics and Cyber Law Structures. These five open research problems are identified at the intersection of certificate security, differentials fuzzing mathematics and new cyber law systems.

Challenge 1: Standardized Certificate Parsing Benchmark.

At present, there is not a standardized benchmark for measuring the effectiveness of differential fuzzing with respect to the parsing of X.509 certificates. Ideally, the Certificate Parsing Benchmark (CPB) would test every one of the 93 conformance requirements of RFC 5280, test all of the documented CWE-295 instances, and test a collection of systematically constructed edge-case certificates in ASN.1 form. [Gap: Methodological]

Challenge 2: D_J to CVSS Base Score Mapping (formal)

A formal relationship has not been articulated in the literature between a D_J value on one hand, and the CVSS exploitability rating on the other. A mapping algorithm between D J [0,1] and CVSS v3.1 scores would significantly help to automate the implementation discrepancies prioritization process, which has been discovered through differential fuzzing. [Gap: Mathematical]

Challenge 3: GDPR Art. 32 Certificate Security Adequacy Standard.

Although Article 32 of the GDPR encourages companies to use state of the art security, the GDPR itself does not refer to best practices in regard to certificate validation. An Adequacy Standard of Technical Adequacy of Certificate Security (TASCS) would transform some values of CCR thresholds, JDR thresholds and minimum fuzzing periods into practices that were Article 32-compliant. [Gap: Regulatory]

Challenge 4: Attribution-Ready Generation of Forensic Artifacts.

The output of modern differential fuzzers does not result in any forensic artifacts that can be effectively used in any tangible form to act against any person legally or otherwise. The following generation of differential fuzzers should offer (a) a description of what exactly about the ASN.1 fields prompting the bugs; (b) CVSS-qualified vulnerability reports; and (c) appropriate chain-of-custody documentation that satisfies the GDPR requirement of reporting data breach within less than 72 hours. [Gap: Tooling & Legal]

Challenge 5: Unremitting Differential Fuzzing in DevSecOps Pipelines.

Each of the fuzzers discussed in this paper is highly configurable and takes much time to execute. To integrate DevSecOps, the issue of decreasing computational overhead must be resolved, and a mechanism of targeting fuzzing to areas where recent code modifications have occurred, and then injecting the results into SBOM management software [14],[33]. [Gap: Engineering]

IX. FUTURE RESEARCH DIRECTIONS

A. Pre-defined Benchmark Data.

A publicly available benchmark data is essential to the domain development. It should contain real-world certificate chains extracted out of real-life deployments of PKI, the results of systematic fuzzing campaigns run on OpenSSL, BoringSSL, WolfSSL and mbedTLS and full test suites of all 93 MUST/SHOULD conformance criteria in RFC 5280 along with the expected acceptance and rejection responses of certificate validators.

B.Blockchain-Certified Certificate Integrity

Blockchain-based PKIs [8], [9] provide audit trails that cannot be tampered with throughout the duration of the certificate generation and revocation process.ration and revocation management. The issue of whether this would make anchoring certificates to the blockchain a creation of an electronic record under Section 65B of the Indian Evidence Act and under GDPR Article 5(f) integrity requirement should be addressed in future studies.

C.Privacy-Saving Federated Certificate Monitoring.

The federated learning algorithms can be used to collectively build the models of anomalous behavior without aggregating the content of the specific certificates - an option that can be essential in cases when the information found in the

certificates can be viewed as sensitive data. Further studies are needed on the techniques of differential privacy federated learning that can satisfy the GDPR Article 25 data minimization criterion.

D. Reinforcement Learning to Fuzz Adaptive Certificates.

Reinforcement learning Fuzzers that can be adapted to new evidence on certificate divergence, previously known classification flaws, and new CVEs reported might generate an adaptive testing system that can cope with new TCEV-2 bugs and with new edge cases of ASN.1 as the RFC documents are revised.

X. CONCLUSION

The Systematic Literature Review has developed the first systematic review ever performed on X.509 certificate parsing exploits that they found by way of differential fuzzing in the domain of software supply chain security. A total of 50 highly influential articles were obtained after the PRISMA 2020 compliant methodology was applied with the help of IEEE Xplore, ACM DL, and ScienceDirect search engines, all published between 2020 and 2025.

The five main contributions of this paper include: (1) The Taxonomy of TCEVs featuring six main categories and 22 sub-categories; (2) Mathematical representation covered in Equations 1–9; (3) Comparison Matrix for 15 contemporary fuzzers showing that deep stateful fuzzers have CCR of 88%–91%; (4) Legal-Technical Mapping that pinpoints gaps in terms of liability in ITA 2000 (§15, 35-39, 43, 66, 66C, 84B/C) and GDPR (Art. 5, 22, 25, 32, 33-34, 82-83); and (5) Open research challenges associated with certificate fuzzing math and emerging cyberspace legislation.

The major finding of this literature review is that the vulnerability surface of the Certificate Parsing is seriously underexplored in practice and poorly protected at large. It should be noted that the technologies required to guard against certificate parsing exploits are already available, but it is not insufficiency that makes them not be used, rather than the fact that poor uptake is an issue.

REFERENCES

[1] H. Chen, S. Kalavar, and N. Bhargava, "Guided, Deep Testing of X.509 Certificate Validation via Coverage Transfer Graphs," in Proc. IEEE ISSRE,2020, DOI: 10.1109/ISSRE5003.2020.9240633.

[2] M. Drashkov, N. Chowdhury, and D. Barrera, "On the Evaluation of X.509 Certificate Processing in TLS Interceptors," in Proc. IEEE TrustCom, 2024, DOI:10.1109/TrustCom64321.2024.10733685.

[3] S. Khan, F. Luo, Z. Zhang, M. A. Rahim, M. Ahmad, and K. Wu, "A Survey on X.509 Public-Key Infrastructure, Certificate Revocation, and Their Modern Implementation on Blockchain," IEEE Commun. Surveys Tuts., vol. 25, no. 4, pp. 2529–2561, Q4 2023, DOI: 10.1109/COMST.2023.3323640.

[4] Y. Chen, J. Lin, H. Zhang, L. Wang, and Y. Liu, "TLS-DeepDiffer: Message Tuples-Based Deep Differential Fuzzing for TLS Protocol Implementations," in Proc. IEEE CSF, 2024, DOI: 10.1109/CSF61742.2024.10589831.

[5] Y. Li, X. Wang, H. Huang, and J. Sun, "Formatted Stateful Greybox Fuzzing of TLS Server," in Proc. IEEE S&P, 2024, DOI:10.1109/SP.2024.10638572.

[6] J. de Ruiter and E. Poll, "Exploiting Dissent: Towards Fuzzing-Based Differential Black-Box Testing of TLS Implementations," IEEE Trans. Dependable Secure Comput., vol. 18, no. 5, pp. 2095–2109, 2021, DOI:10.1109/TDSC.2017.2763962.

[7] M. Luo, B. Feng, L. Lu, E. Kirda, and K. Ren, "On the Complexity of the Web's PKI: Evaluating Certificate Validation of Mobile Browsers," IEEE Trans. Dependable Secure Comput., vol. 21, no. 1, pp. 143–157, 2024, DOI:10.1109/TDSC.2023.3255869.

[8] M. Li, J. Luo, C. Guo, and Z. Yang, "Blockchain-Based Certificate Transparency and Revocation Transparency," IEEE Trans. Dependable Secure Comput., vol. 19, no. 1, pp. 681–697, 2022, DOI: 10.1109/TDSC.2020.2983022.

[9] Z. Xu, Z. Luo, L. Xu, and H. Ma, "ScalaCert: Scalability-Oriented PKI with Redactable Consortium Blockchain Enabled On-Cert Certificate Revocation," in Proc. IEEE ICDCS, 2022, DOI: 10.1109/ICDCS54860.2022.9912265.

[10] L. Wei, J. Wu, C. Long, and B. Li, "An Efficient Blockchain-BasedCross-Domain Authentication and Secure Certificate Revocation Scheme," in Proc. IEEE INFOCOM Workshops, 2021, DOI: 10.1109/INFOCOMWKSHPS51825.2021.9345108.

[11] P. Ladisa, H. Plate, M. Martinez, and O. Barais, "SoK: Taxonomy of Attacks on Open-Source Software Supply Chains," in Proc. IEEE S&P, 2023, DOI: 10.1109/SP46215.2023.10179304.

[12] C. Okafor, T. Scanlon, and J. Williams, "A Landscape Study of Open-Source Tools for SBOM and Supply Chain Security," in Proc. IEEE SecDev, 2025, DOI: 10.1109/SecDev61143.2025.11039284.

[13] T. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks," in Proc. DIMVA, LNCS vol. 12223, Springer, 2020, pp. 23–43.

[14] J. Bi, P. Chen, X. Luo, T. Zhang, and J. Liu, "On the Correctness Of Metadata-Based SBOM Generation: A Differential Analysis Approach," in Proc. IEEE/ACM ICSE, 2024, DOI: 10.1109/ICSE48619.2024.10646983.

[15] N. Zahan et al., "Exploring the Threat of Software Supply

Chain Attacks on Containerized Applications," in Proc. IEEE SANER, 2024, DOI: 10.1109/SANER60148.2024.10474901.

[16] B. Xiao, Y. Xing, N. Shi, W. He, and X. Qi, "Wolves in the Repository: A Software Engineering Analysis of the XZ Utils Supply Chain Attack," in Proc. IEEE/ACM ICSE-SEIP, 2024, DOI: 10.1109/ICSESEIP65941.2024.11025592.

[17] M. Ohm, A. Sykosch, and M. Meier, "Towards Detection of Software Supply Chain Attacks by Forensic Analysis of Install-Time Behavior," in Proc. ACM SIGS ACSAC, 2020.

[18] M. Bohme, V.-T. Pham, and A. Roychoudhury, "Coverage-Based Greybox Fuzzing as Markov Chain," IEEE Trans. Software Eng., vol. 45, no. 5, pp. 489–506, 2019, DOI: 10.1109/TSE.2017.2785841.

[19] H. Li, Z. Zhao, J. Gu, and Y. Fang, "VCDFuzz: A Vulnerability Characteristic Guided Directed Greybox Fuzzing Approach," in Proc. IEEE/ACM ISSTA, 2024, DOI: 10.1109/ISSTA.2024.11160092.

[20] Y. Wang, P. Chen, J. Zhao, and Y. Wei, "Multi-level Directed Fuzzing for Detecting Use-after-Free Vulnerabilities," in Proc. IEEE SANER, 2022, DOI: 10.1109/SANER53432.2022.9724428.

[21] Y. Zhang, W. Hao, P. Chen, and M. Su, "Predecessor-Aware Directed Greybox Fuzzing," in Proc. IEEE/ACM ICSE, 2024, DOI: 10.1109/ICSE.2024.10646882.

[22] K. Huang, J. Xu, X. Li, and Z. Guo, "RumFuzz: Coverage-Guided Greybox Fuzzing with Reasonable Use of Memory," in Proc. IEEE QRS, 2024, DOI: 10.1109/QRS.2024.10684592.

[23] X. Li, Z. Guo, H. Zhang, and F. Wang, "LinFuzz: Program-Sensitive Seed Scheduling Greybox Fuzzing Based on LinUCB Algorithm," IEEE Trans. Dependable Secure Comput., vol. 21, no. 5, pp. 4711–4724, 2024, DOI: 10.1109/TDSC.2024.3391680.

[24] J. Wang, B. Chen, L. Wei, and Y. Liu, "Superion: Grammar-Aware Greybox Fuzzing," in Proc. IEEE/ACM ICSE, 2019, pp. 1174–1185, DOI: 10.1109/ICSE.2019.8811923.

[25] X. Wen et al., "MundoFuzz: Hypervisor Fuzzing with Statistical Coverage Testing and Grammar Inference," in Proc. USENIX Security, 2022.

[26] Y. Gao, L. Zhang, X. Chen, and J. Wang, "Speed is Not All You Need When Fuzzing Stateful Network Servers," in Proc. IEEE S&P, 2024, DOI: 10.1109/SP.2024.11083323.

[27] V.-T. Pham, M. Bohme, A. E. Santosa, A. R. Caciulescu, and A. Roychoudhury, "AFLNET: A Greybox Fuzzer for Network Protocols," in Proc. IEEE ICST, 2020, DOI: 10.1109/ICST46399.2020.9159093.

[28] H. Zhou et al., "Enhancing Coverage in Stateful Protocol Fuzzing via Value-Based Selection," in Proc. IEEE/ACM ICSE, 2024, DOI: 10.1109/ICSE.2024.10733723.

[29] Y. Li et al., "UNIFUZZ: A Holistic and Pragmatic Metrics-Driven Platform for Evaluating Fuzzers," in Proc. USENIX Security, 2021.

[30] T. Petsios, A. Tang, S. Stolfo, A. D. Keromytis, and S. Jana, "NEZHA: Efficient Domain-Independent Differential Testing," in Proc. IEEE S&P, 2017, pp. 615–632, DOI: 10.1109/SP.2017.27.

[31] Z. Guo et al., "Attestation-Based SBOM Integrity Verification for Secur Software Supply Chains," in Proc. IEEE TrustCom, 2025, DOI: 10.1109/TrustCom65204.2025.11181389.

[32] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel, "Small World with High Risks: A Study of Security Threats in the npm Ecosystem," in Proc. USENIX Security, 2019.

[33] X. Xia et al., "An Empirical Study on Software Bill of Materials," in Proc. IEEE/ACM ICSE, 2023, DOI: 10.1109/ICSE48619.2023.00172.

[34] S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey Wolf Optimizer," Advances in Engineering Software, vol. 69, pp. 46–61, 2014.

[35] J. Kennedy and R. Eberhart, "Particle Swarm Optimization," in Proc. IEEE ICNN, 1995, pp. 1942–1948. 8

[36] A. Gupta, S. Juneja, P. Dhiman, A. Koundal, S. Zaguia, and A. Kaur, "Grey Wolf Optimizer with SVM for Hash Integrity Anomaly Detection," in Proc. IEEE ICC, 2022, DOI: 10.1109/ICC45855.2022.9838596.

[37] H. Zhang, Q. Zhao, J. Li, and Y. Wang, "Transformer-PSO Hybrid for E-Contract Hash Anomaly Detection," IEEE Trans. Dependable Secure Comput., vol. 21, no. 4, pp. 2891–2904, 2024.

[38] D. Moher, A. Liberati, J. Tetzlaff, and D. G. Altman, "Preferred Reporting Items for Systematic Reviews: The PRISMA Statement," PLoS Medicine, vol. 6, no. 7, e1000097, 2009.

[39] European Parliament, "General Data Protection Regulation (GDPR), Regulation (EU) 2016/679," Official Journal of the EU, Apr. 2016.

[40] Government of India, "Information Technology Act, 2000, as Amended by IT (Amendment) Act 2008," Ministry of Law and Justice, 2008.

[41] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, "Internet X.509 PKI Certificate and CRL Profile," RFC 5280, IETF, May 2008.

[42] S. Santesson, M. Myers, R. Ankney, A. Malpani, S. Galperin, and C. Adams, "X.509 Internet PKI Online Certificate Status Protocol (OCSP)," RFC 6960, IETF, Jun. 2013.

[43] NIST, "Secure Hash Standard (SHS), FIPS PUB 180-4," National Institute of Standards and Technology, Aug. 2015.

[44] Council of Europe, "Convention on Cybercrime (Budapest Convention)," European Treaty Series No. 185, Nov. 2001.

[45] NIST, "Guide to Integrating Forensic Techniques into Incident Response,"NIST SP 800-86, Aug. 2006.

[46] L. Shen, J. Zhang, W. Xu, and C. Wang, "FIRM-COV: High-Coverage Greybox Fuzzing for IoT Firmware via Optimized Process Emulation," IEEE Trans. Dependable Secure Comput., vol. 20, no. 1, pp. 741–754, 2023, DOI: 10.1109/TDSC.2021.3072039.

[47] M. Muench, J. Stijohann, F. Kargl, A. Francillon, and D. Balzarotti, "What You Corrupt Is Not What You Crash: Challenges in Fuzzing Embedded Devices," in Proc. NDSS, 2018.

[48] P. Chen, Y. Liu, H. Li, and J. Zhao, "Sequence Coverage Directed GreyboxFuzzing," in Proc. IEEE/ACM ICPC, 2019, DOI: 10.1109/ICPC.2019.8813278.

[49] A. Maehren, P. Niemann, R. Merget, D. Bruns, J. Somorovsky, and J. Schwenk, "TLS-Anvil: Adapting Combinatorial Testing for TLS Libraries," in Proc. USENIX Security, 2022.

[50] Q. Wang, P. Chen, J. Zhao, X. Liu, and H. Li, "Multiple Targets Directed Greybox Fuzzing: From Reachable to Exploited," in Proc. IEEE CSF, 2024, DOI: 10.1109/CSF61742.2024.10589744.